Readi World

Engine & Design Bible - Version 0.1

A living technical, visual and gameplay foundation for a modular cozy mobile world.

PURPOSE	This document is the central source of truth for the Readi World project. Separate conversations may design characters, areas, art, systems or code, but they must remain compatible with the decisions recorded here.
LANGUAGE	Development discussion may be Hungarian. In-game text, code, identifiers, file names and technical comments are English.
PRIMARY PLATFORM	iPhone 16 Pro, Safari, landscape orientation, touch-first controls, localhost development with Runestone and a-Shell.
ARCHITECTURE	Single index.html for the prototype, no backend, no framework, no build system. Internally modular and data-driven so it can later be split into files.

Core promise

"A small world that remembers you, visibly grows with you, and always feels good to return to."

Document status: First detailed foundation edition
Created for iterative GPT Project development
Date: 2026-07-19

How to use this Bible

This is a living document, but not every statement has equal flexibility. Every specialized Readi World conversation should begin by treating this Bible as its governing context. Proposed changes should be compared against the project philosophy, technical constraints and existing locked decisions.

Status	Meaning	Rule
LOCKED DECISION	Accepted foundation	Do not silently change. Record a deliberate change in the Decision Log.
RECOMMENDED DEFAULT	Best current choice	May be tuned after testing without redesigning the whole project.
FUTURE SYSTEM	Architecturally reserved	Do not build now unless the roadmap activates it.
OPEN QUESTION	Not yet decided	Prototype, measure and decide later.

Project conversation contract

- All code, IDs, filenames, comments and visible game text are written in English.
- Design discussion may remain Hungarian for speed and clarity.
- New systems must be modular, data-driven and compatible with mobile performance limits.
- A new feature is rejected or simplified if it adds management burden without meaningful play.
- "Coming Soon" is an acceptable and intentional product state.
- The v0.0 prototype proves feeling and architecture; it does not attempt to prove every future system.

Table of contents

CHAPTER 01

Vision and non-negotiable principles

What Readi World is, what it refuses to become, and how every later decision is judged.

LOCKED DECISION	Readi World is a cozy, progression-focused mobile game centered on one evolving home village. It respects player time, avoids forced combat and minimizes inventory administration.

1.1 The emotional target

The intended emotion is ownership without pressure. The player should feel that the village is personally theirs, that even a short visit matters, and that the world remains welcoming rather than demanding. The world should feel alive through motion, sound, light and gradual visual change, not through endless notifications or mandatory chores.

- Two minutes of play should always produce a visible or meaningful result.
- Returning after time away should feel pleasant, never punitive.
- Progress is shown in the environment, not only in numbers and menus.
- The game invites longer sessions but does not punish short sessions.
- Optional challenge may exist, but comfort is the default state.

1.2 Explicit anti-goals

The game must not become	Why
Tower defense	Would redirect the project toward mandatory combat and base protection.
Idle economy simulator	Offline time may advance nature, but should not replace active play.
Heavy farming simulator	Farming is one specialization, not the whole identity.
Inventory management simulator	Resources are categorized and consumed automatically.
Forced-combat RPG	Combat is optional and never gates the peaceful progression path.
Giant open world	Compact areas support focus, performance and meaningful density.

1.3 Decision filter

DESIGN TEST

Before approving any feature, ask: Does this deepen the player experience, or merely increase complexity? Does it respect time? Is it comfortable on a phone? Can it be added without rewriting the engine?

Question	Pass condition
Does it fit the core fantasy?	It strengthens ownership, discovery, visible growth or calm mastery.
Does it work in a two-minute session?	The player can make progress or understand the next goal.
Does it create chores?	If yes, automate or simplify the chore.
Does it force one playstyle?	If yes, create an alternative route or make it optional.
Does it require a new one-off engine?	If yes, first generalize it into reusable data/components.

CHAPTER 02

Player experience and core loop

LOCKED DECISION The player always returns to one home village. Activities expand outward from the hub and feed visible improvements back into it.

2.1 Primary loop

24. Leave the village for a compact activity area or repeatable stage.
25. Gather resources, discover something, complete a small objective or explore.
26. Return home automatically or deliberately.
27. Use resources without manual stack management.
28. Upgrade, decorate or unlock a visible part of the village.
29. See the world change and choose the next specialization.

2.2 Session layers

Session length	Expected value
~2 minutes	Harvest a few resources, check growth, make one small improvement.
~10 minutes	Complete a stage, gather toward an upgrade, explore one route.
~30 minutes	Progress an area tier, build or upgrade, experiment with equipment.
Long session	Chain multiple activities without being forced by timers or energy walls.

2.3 Optional combat

Combat may later add challenge, rare loot and bonus rewards. It must not be required for farming, building, village growth, exploration of peaceful routes or the basic story of belonging. Non-combat players must have a complete progression path.

CHAPTER 03

World structure and scene model

LOCKED DECISION The world is a collection of compact scenes, not one giant continuously loaded map.

3.1 Hub-and-branches model

```
Village Hub
├── Farm
├── Forest stages
├── Mine stages
├── Harbor
│   └── Sea stages
├── Mountains (future)
└── Expedition Gate (future)
```

The village is hand-authored and persistent. Activity areas may be hand-authored, seed-generated, or hybrid. Only the active scene is fully simulated and rendered. Background systems update through elapsed-time calculations rather than keeping every scene alive.

3.2 Recommended logical dimensions

RECOMMENDED DEFAULT Use a 48 x 48 logical-pixel tile for world planning. The renderer may draw sprites larger than one tile. The village begins around 40 x 28 tiles (1920 x 1344 logical pixels).

Scene	Initial planning range	Purpose
Village Hub	40 x 28 tiles	Persistent social and upgrade center.
Farm	24 x 18 to 32 x 24	Compact production area.
Forest stage	24 x 18 to 40 x 28	Repeatable gathering and discovery.
Mine stage	24 x 18 to 32 x 24	Layered resource challenge.
Sea stage	Variable lanes/islands	Navigation, fishing, discovery.

3.3 Area progression

Area	Progression example
Harbor / Sea	Beach -> Shallow Water -> Open Sea -> Deep Ocean -> Islands -> Frozen Sea

Mine	Surface -> Copper -> Iron -> Gold -> Crystal -> Ancient Ruins -> Magic Depths
Forest	Clearing -> Pine Forest -> Old Forest -> Swamp -> Enchanted Grove

Each tier must introduce at least one new meaningful verb, resource, encounter, rule or visual identity. Merely increasing numbers is not sufficient progression.

3.4 Village specialization

LOCKED DECISION

Players choose what their village becomes. A harbor-focused village, mining settlement or agricultural village must all be valid identities. The game must not force equal investment in every branch.

CHAPTER 04

Time, day cycle and offline progression

LOCKED DECISION

The world clock advances in real time while the game is open or closed, but it is not synchronized to the player's local day/night.

4.1 Four-hour world day

Real-time hour	World phase	Visual intention
Hour 1	Morning	Cool warm-up light, dew, waking ambience.
Hour 2	Day	Clear visibility, active village.
Hour 3	Sunset	Warm orange light, long shadows.
Hour 4	Night	Readable moonlit palette, lamps and calm ambience.

After the fourth hour, the cycle returns to morning. This allows players who always play at the same real-world time to experience every phase across sessions.

4.2 Authoritative time model

```
worldTimeMs = savedWorldTimeMs + max(0, nowMs - lastRealTimestampMs)
phase = floor((worldTimeMs % FOUR_HOUR_DAY_MS) / ONE_HOUR_MS)
```

Game systems consume world time, not direct wall-clock time. This allows pausing in debug mode, testing specific phases and later applying server correction without rewriting all systems.

4.3 Offline progression boundaries

Allowed offline	Not allowed offline
Crop growth	Automatic active harvesting
Tree regrowth	Infinite money generation
Construction timers	Quest choices or dialogue decisions
Day/night progression	Combat or dangerous expedition completion
Natural environmental changes	Player movement or resource collection

RECOMMENDED DEFAULT

Cap meaningful offline simulation to 24-48 hours until balancing proves a different limit is healthier. Always show an honest return summary.

4.4 Clock manipulation

Without a backend, phone-time manipulation cannot be fully prevented. The prototype should detect backwards time, unusually large jumps and repeated suspicious patterns, then clamp rewards rather than punish the player. Save integrity matters more than aggressive anti-cheat.

CHAPTER 05

Saving, persistence and recovery

LOCKED DECISION Anything the player changes, wherever they stop, is remembered automatically.

5.1 Save layers

```
save.current
save.backup
save.previousBackup

Each payload contains:
- saveVersion
- gameVersion
- createdAt / updatedAt
- player
- world
- scenes
- progression
- settings
- diagnostics
```

5.2 Save triggers

- After important interactions and upgrades.
- On scene transition.
- At a safe periodic interval, debounced to avoid storage spam.
- When the page becomes hidden or the app enters the background.
- Before destructive operations such as importing another save.
- After migration to a new save version.

5.3 Storage recommendation

For v0.0, IndexedDB is preferred over localStorage for structured saves and future growth. A small emergency pointer or settings fallback may remain in localStorage. The save system should expose one API so storage can later move to cloud sync without touching gameplay systems.

```
SaveManager.commit(reason)
SaveManager.load()
SaveManager.export()
SaveManager.import(payload)
SaveManager.migrate(fromVersion, toVersion)
```

5.4 Validation and recovery

30. Parse the newest save in a protected block.
31. Validate required types, ranges and stable IDs.
32. Repair safe defaults when possible.
33. If validation fails, attempt backupSave.
34. If that fails, attempt previousBackup.

35. Only then offer a new game, while preserving the broken payload for diagnostics.

LOCKED DECISION	Save files are versioned from the first prototype. Stable IDs are never reused for a different meaning.

CHAPTER 06

Engine architecture

LOCKED DECISION The v0.0 ships as one index.html, but its source is organized internally as separable modules and data sections.

6.1 Engine vs game content

```
Engine
├── GameLoop
├── Renderer
├── AtlasManager
├── SceneManager
├── ObjectManager
├── CollisionSystem
├── InteractionSystem
├── InputManager
├── Camera
├── AnimationSystem
├── TimeSystem
├── SaveManager
├── AudioManager
├── EventBus
├── Localization
└── DebugTools

Game Content
├── Village
├── Farming
├── Forest
├── Mining (future)
├── Harbor (future)
├── Fishing (future)
├── Milestones (future)
└── Quests / NPCs (future)
```

6.2 Game loop

```
function frame(now) {
 const dt = clamp((now - lastNow) / 1000, 0, 0.05);
 input.update();
 time.update(dt);
 activeScene.update(dt);
 camera.update(dt);
 renderer.draw(activeScene, camera);
 requestAnimationFrame(frame);
}
```

Delta time is clamped so returning from a suspended browser tab does not simulate a huge physics step. Offline progression is handled separately from frame updates.

6.3 Event-driven boundaries

```
events.emit("tree:hit", { objectId, toolId });
events.emit("resource:added", { resourceId, amount, source });
events.emit("building:upgraded", { buildingId, level });
```

Gameplay systems emit semantic events. Audio, particles, UI notifications, saving and milestones can subscribe without being hardwired into the original action.

CHAPTER 07

World objects, components and state machines

LOCKED DECISION Trees, rocks, crops, buildings, boats and interactables share one general world-object format.

7.1 Base object schema

```
const worldObject = {
  id: "village.harbor.main",
  type: "building",
  subtype: "harbor",
  transform: { x: 820, y: 460, scale: 1 },
  visual: { atlasId: "harbor", state: "normal", level: 2 },
  collision: { enabled: true, shape: "box", ... },
  interaction: { type: "upgrade", range: 72 },
  audioProfile: "harbor",
  persistent: true,
  tags: ["village", "waterfront"]
};
```

7.2 Components

Component	Responsibility
Transform	Position, scale, facing and depth anchor.
Visual	Atlas, cell, animation, tint and visibility.
Collision	Physical footprint; independent from full sprite bounds.
Interaction	Action label, range, requirements and callback ID.
Health/Harvest	Hits, resource yield, damage state and regeneration.
Growth	Stage, timestamps and next transition.
Upgrade	Level, costs, construction state and unlocks.
Audio profile	Semantic sound hooks, not direct file paths.
Persistence	What state is serialized.

7.3 State machines

Entity	Valid states

Player	idle, walking, interacting, usingTool, fishing, transitioning, disabled
Tree	healthy, damaged, falling, stump, regrowing
Building	locked, available, constructing, active, upgrading
Crop	empty, planted, growing, ready, withered (optional)
Stage	locked, available, active, cleared, mastered

Transitions must be explicit. Invalid combinations such as walking while fishing are prevented by the state machine, not repaired afterward by scattered flags.

CHAPTER 08

Rendering, layers, atlas and pivots

LOCKED DECISION The entire visual pipeline is atlas-based, with standardized cells, pivots, depth anchors and metadata.

8.1 World render layers

```
0  Ground
10 Paths and soil
20 Ground decoration
30 Low objects and crops
40 Buildings and trunks
50 Characters and moving objects
60 Canopies / foreground occlusion
70 Weather and world effects
80 Interaction highlights
90 World-space labels
100 UI
```

Within depth-sensitive layers, draw order is derived from each object's foot/pivot Y coordinate. A tall tree may visually cover a character while its collision remains only around the trunk base.

8.2 Atlas metadata

```
const atlas = {
 image: "assets/trees.png",
 frameWidth: 192,
 frameHeight: 256,
 columns: 4,
 rows: 4,
 sprites: {
  "tree.oak.a.healthy": { col: 0, row: 0, pivot: [0.5, 0.92] },
  "tree.oak.a.damaged": { col: 1, row: 0, pivot: [0.5, 0.92] }
 }
};
```

8.3 Atlas grouping rule

- Group assets with similar physical scale and purpose.
- Do not create one enormous atlas containing tiny icons and huge buildings.
- Use separate atlases such as trees, plants, rocks, houses, harbor, boats and UI icons.
- Never draw grid lines, labels or numbers into production PNG atlases.
- All cells include transparent safety padding and a consistent ground line.

8.4 Placeholder behavior

Missing assets never produce a silent blank or black screen. The renderer draws a visible magenta-style diagnostic placeholder with the missing asset ID in debug mode, and a neutral fallback in player mode.

CHAPTER 09

Character, clothing, tools and animation

LOCKED DECISION

The character is assembled from synchronized visual layers rather than a unique full sprite sheet for every outfit and tool combination.

9.1 Layer stack

```
shadow
body_base
legs / pants
boots
torso / shirt / armor
head
hair_back / hair_front
held_tool_or_weapon
hat / accessory
foreground_effect
```

Every layer uses the same frame grid, direction order, pivot and animation timing. Changing a shirt or axe swaps only the relevant atlas cell, preserving the base animation.

9.2 Character atlas convention

Rows	Meaning
0	Walk / idle facing down
1	Walk / idle facing left
2	Walk / idle facing right
3	Walk / idle facing up

Columns	Meaning
0-5	Animation frames for the selected action
Unused cells	Transparent and reserved, never filled inconsistently

RECOMMENDED DEFAULT

Prototype character cells: 128 x 128. Final production size may change after the style test, but every character layer must share exactly the same dimensions.

9.3 Equipment data

```
player.appearance = {
  bodyId: "body.base.01",
```

```
  hairId: "hair.short.03",
  shirtId: "shirt.farmer.01",
  pantsId: "pants.work.01",
  bootsId: "boots.leather.01",
  hatId: "hat.straw.01",
  equippedToolId: "tool.axe.copper"
};
```

9.4 Tool animation compatibility

Tools declare which animation profiles they support, such as swing, mine, fish or water. The player animation owns body movement; the tool layer owns the synchronized visual. A new iron axe therefore requires asset/data additions, not a new character controller.

9.5 Z-order exceptions

Some layers change order by direction. For example, a tool may render behind the body when facing up and in front when facing down. This must be metadata-driven per animation profile.

CHAPTER 10

Village, buildings, boats and visual progression

LOCKED DECISION Village progression must be visually obvious. Building level is represented by relevant atlas cells and optional additive layers.

10.1 Building strategies

Strategy	Use when	Example
Full-state atlas cell	The whole silhouette changes substantially	Harbor Lv1-Lv4
Layered building	Base remains stable while parts change	House base + roof + chimney + decoration
Hybrid	Major levels replace base; cosmetics are layered	Boat hull level + sail + flag + cargo

10.2 Harbor example

Level	Visible identity	Functional direction
Lv1	Small wooden pier, worn boat	Basic shoreline access
Lv2	Longer pier, crates, lantern	Improved fishing and storage
Lv3	Loading crane, market, sailboat	Trade and open-sea access
Lv4	Warehouse, multiple ships, strong lighting	Advanced routes and specialization

10.3 Construction transition

36. The object enters constructing/upgrading state.
37. Temporary scaffolding, wood piles or dust particles appear.
38. Progress continues according to world time if appropriate.
39. The atlas state switches only after completion is committed to the save.
40. A short reveal animation, sound event and optional milestone feedback play.

10.4 Coming Soon landmarks

Future systems appear as intentionally designed locations in the village: a blocked mine entrance, unfinished harbor, sealed expedition gate or mountain path. They display "Coming Soon" and never behave like broken buttons. This communicates the roadmap while keeping development focused.

CHAPTER 11

Procedural generation and stage design

LOCKED DECISION Procedural decoration is seed-based and rule-driven, never uncontrolled randomness.

11.1 Deterministic random streams

```
worldSeed
├── vegetationSeed
├── rockSeed
├── weatherSeed
├── villageDecorationSeed
└── stageSeed(areaId, stageIndex, attemptIndex)
```

The engine never uses Math.random() for persistent generation. Sub-seeds prevent a flower-system update from rearranging all trees.

11.2 Forest placement rules

Rule	Example default
Species weights	55% oak, 25% pine, 10% young tree, 5% bush, 5% stump
Minimum spacing	Trees respect trunk distance, not canopy bounds.
Clear paths	No harvestable object blocks navigation routes.
Building clearance	Maintain interaction space around doors and landmarks.
Ecological clustering	Pines cluster, reeds prefer water, flowers favor clearings.
Density zones	Edges may be denser; central paths remain readable.

11.3 Persistent world generation

A persistent scene is generated once and stores its seed plus changed object states. Revisiting the area reproduces the same layout. Only harvested, moved, upgraded or regrown objects differ.

11.4 Repeatable stages

Forest, mine and sea stages may be replayed. A stage definition contains biome rules, objectives, resource table, size range, progression tier and a deterministic seed. Replaying may use the same layout for mastery or a new attempt seed for variety, depending on the stage type.

11.5 Generator debug panel

- Tree density slider
- Bush density slider
- Pine percentage slider
- Clearing size slider
- Regenerate
- Copy seed
- Accept layout
- Show exclusion zones

CHAPTER 12

Input, interaction, camera and mobile UI

LOCKED DECISION The game is touch-first and landscape-first, but gameplay listens to abstract actions rather than raw touch coordinates.

12.1 Input actions

```
Input.moveVector
Input.interactPressed
Input.useToolPressed
Input.openMenuPressed
Input.cancelPressed
Input.debugTogglePressed
```

12.2 Recommended mobile layout

- Left side: virtual joystick or movable touch region.
- Right side: one large context-sensitive action button.
- Optional secondary tool button only when testing proves it is needed.
- Top safe area: time phase, energy/stamina if retained, currency and menu.
- All controls respect iOS safe-area insets and Dynamic Island avoidance.
- Left-handed layout is available in settings.

12.3 Context interaction

```
{
 interactionType: "harvest",
 actionLabelKey: "action.chop",
 range: 64,
 requiredToolTag: "axe",
 priority: 40
}
```

The InteractionSystem finds the best valid nearby target based on distance, facing, priority and line of access. It highlights that target and updates the action label to Chop, Mine, Fish, Talk, Enter or Upgrade.

12.4 Camera

- Smoothly follows the player with gentle damping.
- Clamps to scene boundaries and never reveals empty outside space.
- Uses a logical coordinate system independent of device pixels.
- Optional camera shake is subtle and can be disabled.
- Extra-wide screens show slightly more world but do not expose hidden mechanics or create competitive advantage.

CHAPTER 13

Resources, inventory, economy and milestones

LOCKED DECISION Inventory is automatic and categorized. Players do not manually move stacks between chests.

13.1 Resource storage

```
inventory.resources = {
  "resource.wood.common": 42,
  "resource.stone.common": 18,
  "resource.fish.bluegill": 3
};
```

Crafting and construction query the central resource service and consume materials through a validated transaction. The player sees understandable totals and shortages, not storage logistics.

13.2 Economic rules

- Begin with one ordinary currency plus resources.
- Avoid unnecessary premium or regional currencies.
- No mandatory energy wall that stops play.
- Energy, if used, modifies efficiency or pacing rather than forbidding activity.
- Upgrade costs grow meaningfully, not exponentially for its own sake.
- The player never loses multiple days of progress as a punishment.
- Monetization must never deliberately create frustration that payment then removes.

13.3 Central transactions

```
economy.addCurrency("coin", 100, { source: "stage_reward" });
inventory.add("resource.wood.common", 3, { source: "tree.harvest" });
upgradeService.purchase("building.harbor", 2);
```

13.4 Milestones

Traditional achievements become calm milestones that celebrate natural play. They may grant decorations, cosmetics, titles or small currency rewards without creating a compulsory checklist.

Example ID	Display text	Possible reward
milestone.first_harvest	First Harvest	Farm sign decoration
milestone.forest_friend	Forest Friend	Leaf-pattern shirt
milestone.growing_village	Growing Village	Village banner
milestone.across_the_water	Across the Water	Boat flag

CHAPTER 14

Audio-ready architecture

LOCKED DECISION Detailed sound design may come later, but every system is audio-ready from v0.0.

14.1 Semantic audio events

```
events.emit("audio:request", {
  profile: "tree.hit",
  material: "wood",
  toolTier: "copper",
  position: { x, y }
});
```

Gameplay code never directly references a specific mp3 filename. Audio profiles later choose randomized variants, volume, pitch range, distance falloff and cooldown.

14.2 Mixer groups

Group	Examples
Master	Global volume
Music	Future soundtrack
Ambience	Wind, birds, water, night insects
Effects	Tools, footsteps, building
UI	Buttons, notifications

14.3 Mobile browser rule

Safari requires user interaction before audio playback. AudioManager initializes silently and unlocks on the first valid tap, while preserving settings and avoiding unexpected sound on load.

CHAPTER 15

Performance, reliability and accessibility

LOCKED DECISION The phone must remain cool, responsive and battery-conscious. Visual ambition never justifies unstable play.

15.1 Performance budget

Area	Target
Frame rate	60 FPS target; graceful 30 FPS fallback if required
Frame time	~16.7 ms target at 60 FPS
Active scene	Only current scene fully updated
Device pixel ratio	Clamp render scale, recommended max 2
Draw calls	Cull objects outside camera margin
Allocations	Avoid creating arrays/objects every frame
Atlas loading	Load once and reuse cached ImageBitmap/Image

```
const renderScale = Math.min(window.devicePixelRatio || 1, 2);
```

15.2 Reliability

- Central error boundary for startup and scene loading.
- Missing assets use fallback graphics instead of stopping the game.
- Invalid objects are logged and skipped where safe.
- Save validation occurs before replacing the last known good backup.
- A debug-safe mode can disable particles, procedural decoration and optional systems.

15.3 Accessibility defaults

- Adjustable UI scale and readable minimum text size.
- Important information is never communicated by color alone.
- Camera shake can be disabled.
- Reduced motion mode lowers particles and decorative animation.
- Separate volume controls for music, ambience, effects and UI.
- Left-handed control layout.
- Large touch targets with visible pressed states.
- Contrast checks for day and night palettes.

CHAPTER 16

Localization and naming rules

LOCKED DECISION The shipped game and all technical artifacts are English-first. Hungarian is the design discussion language only.

16.1 Stable IDs

```
building.harbor
tree.oak.common
tool.axe.copper
area.forest.stage_01
milestone.first_harvest
ui.action.coming_soon
```

Stable IDs are lowercase, dot-separated and semantic. Display names may change; IDs do not. Never reuse an old ID for a different concept.

16.2 Localization keys

```
const en = {
 "building.harbor.upgrade": "Upgrade Harbor",
 "area.mine.comingSoon": "Mine - Coming Soon",
 "action.chop": "Chop"
};
```

16.3 File naming

- Use lowercase kebab-case for files: forest-trees.png, character-tools.png.
- Use camelCase for JavaScript variables and functions.
- Use PascalCase for classes or manager modules.
- Use SCREAMING_SNAKE_CASE for true constants.
- No spaces, accents or Hungarian words in technical filenames.

CHAPTER 17

Security, anti-tamper and future backend

LOCKED DECISION A fully client-side HTML game cannot be made truly cheat-proof. The goal is resilient architecture, not false security claims.

17.1 v0.0 protection

- Validate save schemas and numeric ranges.
- Use checksums to detect accidental corruption and casual editing.
- Keep rolling backups.
- Detect suspicious time jumps and clamp offline rewards.
- Route all economy changes through central services.
- Record transaction sources for diagnostics.
- Separate game logic from rendering so a future server can become authoritative.

17.2 Do not overbuild now

- Aggressive obfuscation
- Complex encryption with client-stored keys
- Device-locked saves
- Custom account system
- Heavy DRM
- Continuous anti-cheat polling

17.3 Backend trigger points

Feature	Why backend becomes necessary
Real-money purchases	Server/store receipt verification
Premium currency	Authoritative balance and fraud prevention
Cloud save	Conflict resolution and identity
Leaderboards	Trusted scores
Player trading	Duplication and fraud prevention
Multiplayer/shared events	Authoritative world state

17.4 Payment boundary

Payment is intentionally excluded from v0.0. The economy and entitlement APIs should nevertheless be designed so a later web, PWA or App Store wrapper can verify purchases externally and then grant entitlements through a single trusted interface.

CHAPTER 18

Art direction and asset pipeline

RECOMMENDED DEFAULT	Use a hand-painted, clean 2D style with soft 3D volume, warm colors, gentle shadows and no harsh pixel-art constraint. Final direction is locked only after a style test.

18.1 Visual pillars

- Top-down with a lightly tilted perspective.
- Readable silhouettes at phone size.
- Warm, friendly palette with controlled saturation.
- Soft shadows that share one light direction.
- Consistent scale, ground line and camera angle.
- Enough detail to feel alive, never enough to become noisy.
- Night remains playable and readable.

18.2 Art Bible reference set

41. Hero image showing the entire prototype village and connected zones.
42. Morning, day, sunset and night versions of the same scene.
43. Master color palette and material swatches.
44. Character proportion and silhouette sheet.
45. Forest atlas test with multiple tree variants.
46. Village building progression test, especially harbor levels.
47. UI panel, button and icon style sample.
48. Lighting, shadows, water and foliage motion rules.

18.3 Asset production pipeline

49. Write an asset brief with dimensions, grid, perspective, palette and purpose.
50. Generate or draw a concept sheet using the current style reference.
51. Reject inconsistent face, proportions, light direction or ground line.
52. Prepare transparent production PNG without labels or grid marks.
53. Record atlas metadata, pivots and collision footprint.
54. Test in the live engine at actual phone scale.
55. Approve, version and add to the asset registry.

18.4 AI consistency rule

The greatest risk is not insufficient art but inconsistent art. Every prompt must reference the same visual vocabulary, camera, proportions, palette, light direction, cell dimensions and transparent-background rules. Unrelated one-off images should not become production assets.

CHAPTER 19

Development workflow, testing and roadmap

LOCKED DECISION Build one complete system at a time. Unfinished areas show Coming Soon rather than half-working mechanics.

19.1 Development order

56. Prove the general engine system with placeholders.
57. Make one interaction work end-to-end.
58. Add saving and reload verification.
59. Improve visual and tactile feedback.
60. Turn content into data, not duplicated code.
61. Only then expand variants and new areas.

19.2 Roadmap

Version	Scope
0.0	Movement, camera, village scene, forest interaction, atlas demo, day cycle, autosave, Coming Soon landmarks, debug panel.
0.1	Farming core, basic resources, first visible village upgrade.
0.2	Mine entrance and first mining stage.
0.3	Harbor progression and fishing foundation.
0.4	Boat and sea-stage navigation.
0.5	Expedition framework and optional challenge systems.
1.0	Coherent release-quality core experience, not necessarily every future area.

19.3 Testing checklist

- Start fresh and reach the prototype goal without developer tools.
- Reload after every important action and confirm exact state restoration.
- Background Safari, wait, return and verify time progression.
- Rotate or resize and confirm safe UI behavior.
- Test both touch sides and left-handed mode.
- Run with missing asset IDs to verify fallbacks.
- Test low-power mode and a 30 FPS fallback.
- Copy a seed and reproduce the exact world.

- Import an older saveVersion and verify migration.
- Inspect memory/performance after repeated scene transitions.

CHAPTER 20

Readi World v0.0 prototype specification

The first playable vertical slice: small, beautiful, persistent and architecturally honest.

PROTOTYPE GOAL	When the game opens, it should already feel pleasant to stand in the world. The prototype proves movement, visual identity, persistence, time and modular systems - not content volume.

20.1 Playable content

- One compact village hub with a house, central path, farm edge and forest entrance.
- A visible harbor, mine and expedition landmark marked Coming Soon.
- One player character with four-direction movement and layered placeholder appearance.
- Several tree variants from one atlas.
- At least one harvestable tree with healthy, damaged, stump and regrowing states.
- Automatic wood storage and one simple visible upgrade or build action.
- Four-phase world day that continues while the game is closed.
- Autosave, backup, reload and debug export/import.
- Ambient sound architecture with at least minimal prototype ambience/effects when assets exist.
- Seeded decoration and a debug regeneration tool.

20.2 Minimal UI

- Current time phase
- Wood amount
- Coin amount if used
- Context action button
- Small menu/settings button
- Optional unobtrusive debug toggle

20.3 Prototype success criteria

Criterion	Pass
Feel	Movement, camera and world ambience feel calm and responsive.
Persistence	Reloading restores position, object states, resources and time.
Architecture	A new tree or building level can be added through data and atlas cells.
Performance	Stable on iPhone 16 Pro without excessive heat or visible stutter.

Clarity	Coming Soon areas look intentional, not broken.
Scalability	Forest, mine and harbor can later plug into SceneManager and shared systems.

20.4 Explicit exclusions

- Full farming simulation
- NPC schedules and dialogue
- Quest system
- Combat
- Real-money payments
- Backend/cloud account
- Complete music library
- Large inventory UI
- Multiple finished regions

20.5 Suggested index.html organization

```
<!-- 1. HTML shell and canvas -->
<style>/* 2. UI and responsive layout */</style>
<script>
// 3. Constants and configuration
// 4. Utility functions and seeded RNG
// 5. EventBus and managers
// 6. Renderer, atlas and animation
// 7. World object/component systems
// 8. Scene data and procedural generator
// 9. Player, input, interaction and camera
// 10. Time, save and audio
// 11. UI and debug tools
// 12. Bootstrap and game loop
</script>
```

CHAPTER 21

Appendix A - Reference data schemas

A.1 Save root

```
{
 saveVersion: 1,
 gameVersion: "0.0.0",
 timestamps: { createdAt, updatedAt, lastRealTime },
 player: { sceneId, position, appearance, equippedToolId },
 world: { seed, worldTimeMs, unlockedAreaIds },
 scenes: { [sceneId]: sceneState },
 inventory: { resources, currency },
 progression: { buildings, areaTiers, milestones },
 settings: { audio, uiScale, handedness, reducedMotion },
 diagnostics: { lastSaveReason, checksum }
}
```

A.2 Atlas registry

```
{
 id: "trees.forest.01",
 src: "assets/forest-trees.png",
 frameWidth: 192,
 frameHeight: 256,
 columns: 4,
 rows: 4,
 entries: {
  "tree.oak.a.healthy": { col: 0, row: 0, pivot: [0.5,0.92], collision: [...] }
 }
}
```

A.3 Stage definition

```
{
 id: "area.forest.stage_01",
 biomeId: "biome.forest.clearing",
 size: { widthTiles: 28, heightTiles: 20 },
 generatorProfileId: "forest.clearing.basic",
 objectives: [],
 resourceTableId: "loot.forest.stage_01",
 unlockRequirements: [],
 repeatMode: "new_attempt_seed"
}
```

CHAPTER 22

Appendix B - AI prompt standards

B.1 Asset prompt template

```
Create a production-ready 2D sprite atlas for Readi World.

STYLE:
- warm hand-painted cozy mobile game art
- clean readable silhouettes
- soft 3D volume, no harsh outlines
- top-down lightly tilted camera
- consistent upper-left light direction
- match the approved Readi World palette and reference sheet

TECHNICAL:
- transparent background
- exact [COLUMNS] x [ROWS] grid
- each cell exactly [WIDTH] x [HEIGHT] px
- identical ground line and pivot across cells
- safety padding inside every cell
- no labels, numbers, grid lines or shadows outside cells

CONTENT MAP:
Row 0: ...
Row 1: ...

CONSISTENCY:
All variants must look like the same asset family, with identical perspective, scale, light and material treatment.
```

B.2 Character-layer prompt rule

Every clothing, hair, hat, boot and tool sheet must use the approved base-character sheet as a visual reference. It must preserve exact cell dimensions, pose, frame timing, limb placement, pivot and direction order. The generated layer must contain only the requested layer on transparency.

B.3 Environment hero image prompt goals

- Show the compact hub structure clearly
- Include farm, forest, harbor, mine and expedition landmarks
- Demonstrate paths and readable navigation
- Establish scale between character, tree and building
- Provide lighting and material reference
- Avoid adding mechanics not approved in this Bible

CHAPTER 23

Appendix C - Decision log and open questions

C.1 Locked decisions recorded in v0.1

ID	Decision
D-001	One persistent village hub with compact connected scenes.
D-002	Touch-first, landscape-first iPhone development.
D-003	Single index.html prototype with modular internal architecture.
D-004	Atlas-based visual system for characters, environment, buildings and boats.
D-005	Layered character equipment and clothing.
D-006	Automatic categorized inventory.
D-007	Optional combat; peaceful progression remains complete.
D-008	Four-hour real-time world day independent of local day/night.
D-009	Automatic persistent saving of player and world state.
D-010	Seeded deterministic generation and persistent layouts.
D-011	English game/code/IDs; Hungarian design discussion allowed.
D-012	Audio-ready semantic event system from v0.0.
D-013	Coming Soon is an intentional development and UX state.
D-014	Security is resilient but not falsely claimed cheat-proof without backend.

C.2 Recommended defaults to validate in v0.0

- 48 x 48 logical planning tile.
- Village around 40 x 28 tiles.
- Character cells around 128 x 128.
- Canvas device-pixel-ratio clamp at 2.
- 24-48 hour offline-progression cap.

- Virtual joystick plus one context action button.
- Hand-painted 2D style with soft depth rather than strict pixel art.

C.3 Open questions

ID	Question	When to decide
Q-001	Exact final art style and palette	After first hero-image/style test
Q-002	Joystick vs tap-to-move as default	During v0.0 feel testing
Q-003	Whether energy exists at all	After core loop testing
Q-004	Exact village dimensions	After camera and navigation prototype
Q-005	Persistent vs regenerated layout per stage type	During forest-stage prototype
Q-006	Offline progression cap	After economy timing tests
Q-007	Web/PWA/App Store release route	Before monetization or public release
Q-008	Cloud save and account strategy	Before cross-device support

C.4 Change procedure

62. State which locked decision is being challenged.
63. Explain the player benefit and technical cost.
64. Describe migration impact on saves, assets and code.
65. Prototype the change if uncertainty is experiential.
66. Record the approved outcome as a new Decision Log entry.
67. Update all affected chapters and increment the Bible version.

Readi World v0.1 Foundation

Build the system once. Let the world grow through data, art and meaningful choices.

NEXT ACTION	Use Chapter 20 as the implementation brief for Readi World v0.0. Before production asset generation, create the first approved visual reference set described in Chapter 18.